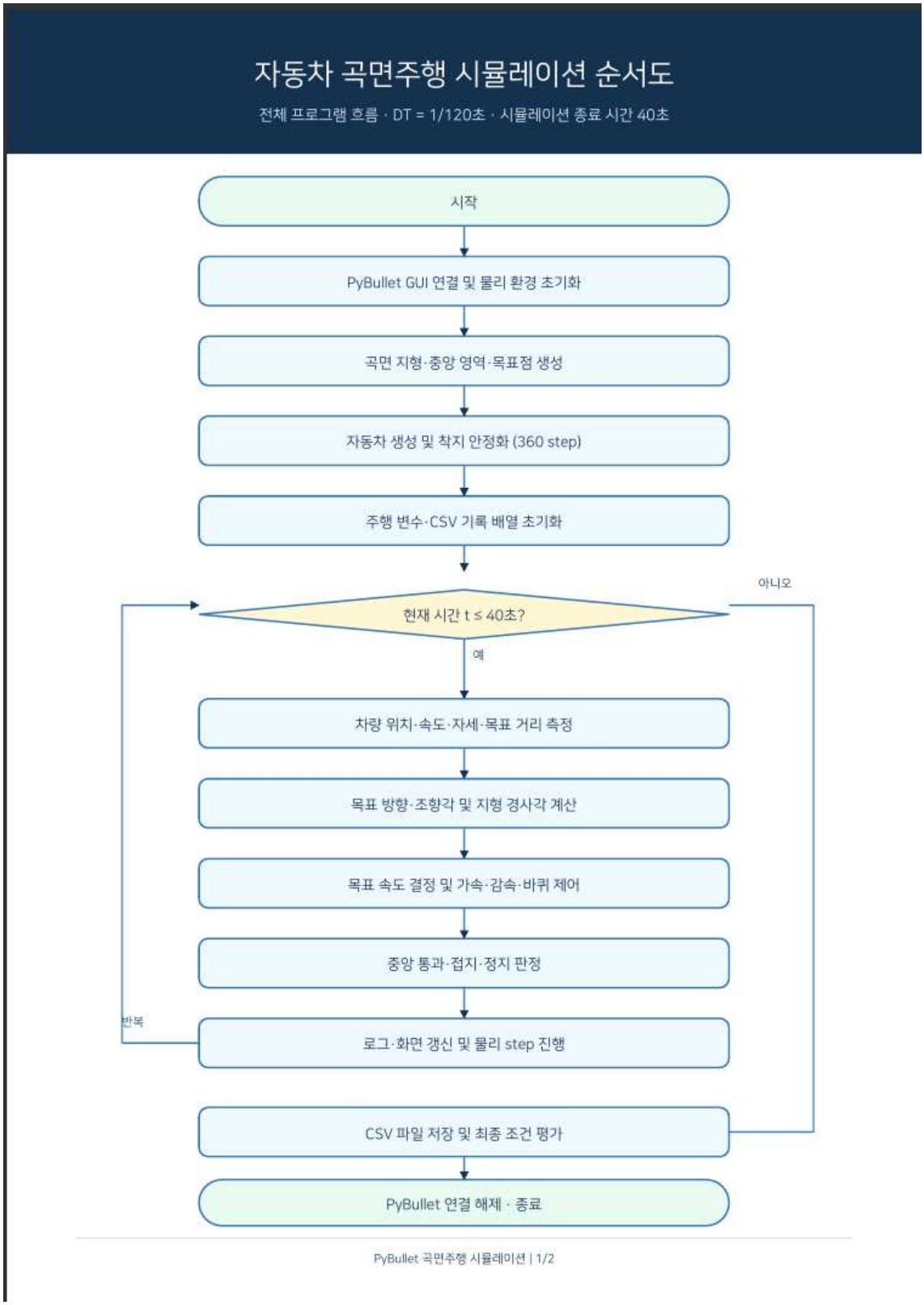
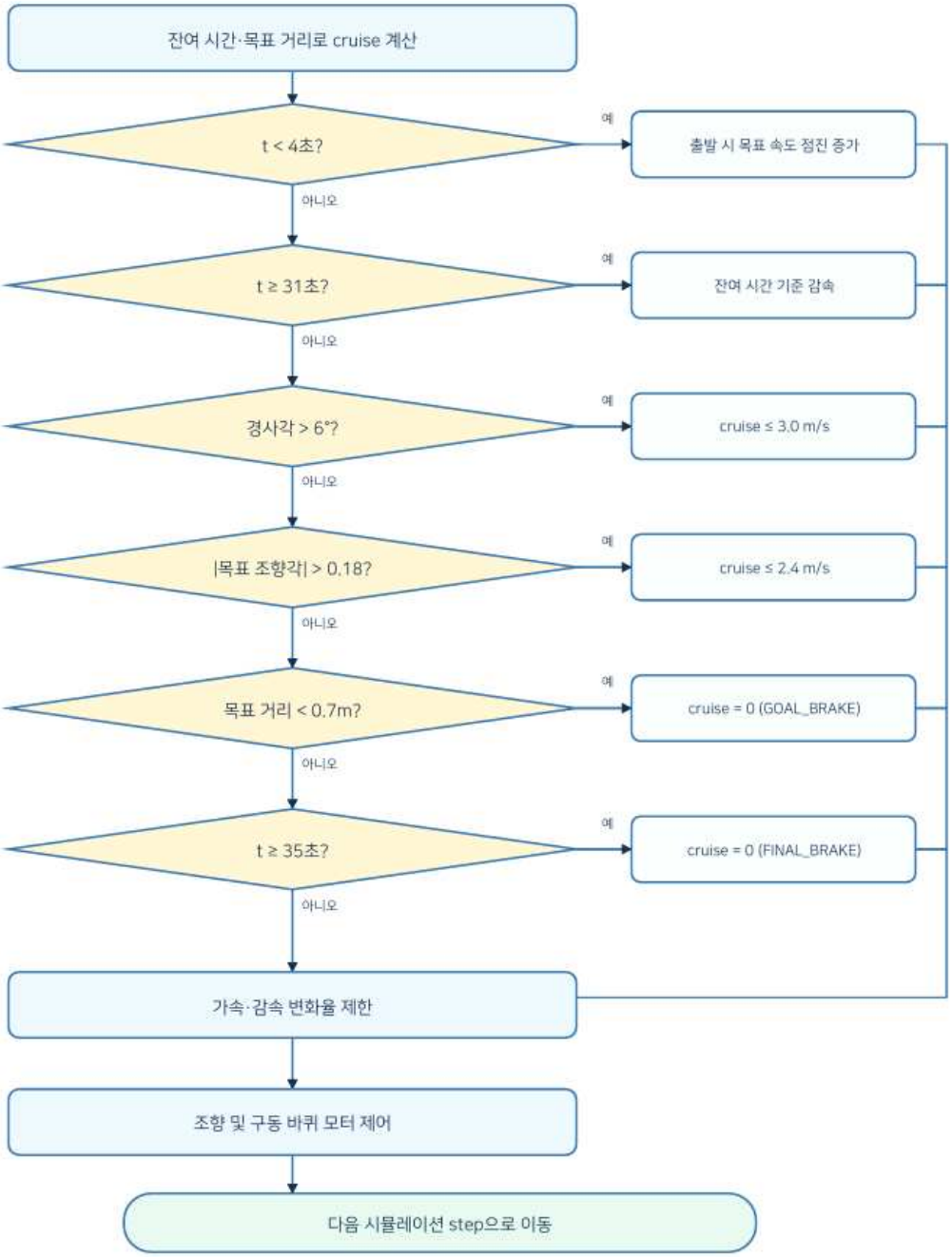
<경사 곡면의 자율주행 시뮬레이션>

202502552 이현준



주행 속도 제어 세부 순서도

코드의 조건문 순서에 따른 속도 제한 및 정지 제어



참고: 각 제한 조건은 순서대로 검사되며, 후속 조건이 이전 속도 제한을 다시 변경할 수 있습니다.

1. 시뮬레이션 개요

본 시뮬레이션은 PuBullet 물리 엔진을 이용하여 곡면 지형에서 자동차가 목표 지점까지 자동으로 주행하고 정지하도록 구현한 프로그램이다.

자동차는 현재 위치와 목표 위치를 비교하여 주행 방향을 결정하고, 지형의 경사각과 조향각에 따라 목표 속도를 조절한다. 또한 급격한 가속과 감속을 방지하기 위해 속도 변화율을 제한한다.

시뮬레이션은 120Hz로 물리 계산을 수행하며 35초 이휴에는 최종 정지 명령을 적용한다. 전체 시뮬레이션은 40초까지 진행하고, 주행 과정에서 측정한 데이터를 csv 파일로 저장한다.

2. 전체 순서도 및 코드 설명

① 시작 및 PuBullet 환경 초기화

프로그램이 실행되면 main() 함수가 호출되고 PyBullet GUI 환경을 생성한다.

```
DT = 1.0 / 120.0
DRIVE_END = 35.0
DISPLAY_END = 40.0

cid = p.connect(p.GUI)
p.setGravity(0, 0, -9.81)
p.setTimeStep(DT)
```

물리 계산의 시간 간격은 1/120초로 설정하며, 중력가속도는 -9.81m/s^2를 적용한다. 자동차의 주행 제어는 35초를 기준으로 종료하고, 시뮬레이션은 40초까지 유지한다.

② 곡면 지형 및 목표 영역 생성

코드에서 OBJ 파일을 사용하지 않고, 수학적인 높이 함수를 이용하여 곡면 지형을 생성한다.

$$z(x, y) = Asin(kx)\cos(ky)$$

여기서 $A = 1.8, k = \frac{2\pi}{32}$이다.

```
def height(x, y):
    k = 2.0 * math.pi / WAVELENGTH
    return A * math.sin(k * x) * math.cos(k * y)
```

100m x 100m 영역을 201 x 201개의 격자로 나누어 Heightfield 지형을 생성한다.

이후 중앙 영역의 경계와 목표 지점을 시각적으로 표시한다.

- 시작 위치(3, 3)m
- 목표 위치: (48, 48)m
- 중앙 영역: X, Y 좌표가 각각 42~58m인 영역

③ 자동차 생성 및 착지 안정화

PyBullet에서 제공하는 racecar.urdf 모델을 불러와 자동차를 생성한다.

```
car = p.loadURDF(
    "racecar/racecar.urdf",
    [START[0], START[1], height(*START) + 1.3],
    quat,
    globalScaling=CAR_SCALE
)
```

자동차의 크기는 기본 모델의 2.5배로 설정한다.

이후 360회의 물리 계산을 수행하여 자동차가 지형에 안정적으로 착지하도록 한다.

```
for _ in range(360):
    p.stepSimulation()
```

이 단계는 본격적으로 주행이 시작되기 전에 차량의 초기 상태를 안정화하기 위한 과정이다.

④ 주행 변수 초기화 및 반복문 실행

주행을 시작하기 전에 조향각, 목표 속도, 이동 거리, 최대 롤각, 정지 시간 등의 변수를 초기화한다.

```
smooth_steer = 0.0
smooth_speed = 0.0

passed_at = None
stopped_at = None
stop_since = None

max_roll = 0.0
total_distance = 0.0
```

이후 다음 반복문을 실행한다.

```
for step in range(int(DISPLAY_END / DT) + 1):
    t = step * DT
```

각 반복문에서 현재 위치와 속도를 측정하고, 목표 방향과 목표 속도를 계산한 후 자동차의 바퀴를 제어한다.

반복문은 40초에 해당하는 마지막 step까지 실행된다.

⑤ 현재 위치, 속도 및 자세 측정

자동차의 위치, 회전 자세, 속도를 PyBullet에서 읽어온다.

```
pos, orn = p.getBasePositionAndOrientation(car)
vel, _ = p.getBaseVelocity(car)

roll, pitch, yaw = p.getEulerFromQuaternion(orn)
speed = float(np.linalg.norm(vel))
```

현재 위치와 목표 위치 사이의 거리는 다음 식으로 계산한다.

$$d = \sqrt{(x_g - x)^2 + (y_g - y)^2}$$

이때 계산한 거리는 목표점 접근에 따른 속도 제어와 최종 위치 오차 판정에 사용한다.

⑥ 목표 방향 및 조향각 계산

현재 자동차의 위치에서 목표 지점을 향하는 방향각을 계산한다.

$$\theta_{target} = \text{atan2}(y_g - y, x_g - x)$$

자동차의 현재 진행 방향인 yaw와 목표 방향의 차이를 이용하여 조향각을 결정한다.

```
heading = math.atan2(
    GOAL[1] - xy[1],
    GOAL[0] - xy[0]
)

err_heading = wrap(heading - yaw)

target_steer = float(
    np.clip(1.6 * err_heading, -MAX_STEER, MAX_STEER)
)
```

목표 조향각은 ± 0.35rad 범위로 제한하며, move_toward() 함수를 이용하여 조향각이 급격하게 변하지 않도록 제어한다.

⑦ 경사각 및 목표 속도 계산

곡면 지형의 높이 함수에 대해 X축과 Y축 방향의 편미분을 계산한다.

$$\frac{\partial z}{\partial x} = Ak\cos(kx)\cos(ky)$$

$$\frac{\partial z}{\partial y} = -Ak\sin(kx)\sin(ky)$$

이를 이용하여 지형의 경사각을 계산한다.

$$\alpha = \tan^{-1}\left(\sqrt{\left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2}\right)$$

코드에서는 이 값을 도 단위로 변환한다.

```
dzdx, dzdy = gradient(xy[0], xy[1])

slope = math.degrees(
    math.atan(math.hypot(dzdx, dzdy))
)
```

목표 속도는 목표점까지의 거리와 남은 주행 시간을 기준으로 계산한다.

```
remaining_time = max(0.1, DRIVE_END - t)

cruise = min(
    MAX_SPEED,
    distance / remaining_time * 1.15
)
```

따라서 목표점까지의 거리가 줄어들거나 남은 시간이 달라지면 목표 속도도 변화한다.

⑧ 목표 속도 제한 조건 적용
이 단계는 순서도 2페이지에 해당하며, 코드에 작성된 조건문의 순서대로 목표 속도를 변경한다.

(1) 주행 시간 4초 미만: 출발 시 목표 속도를 점진적으로 증가
(2) 주행 시간 31초 이상: 목표 거리와 남은 시간을 이용해 감속
(3) 경사각 6도 초과: 목표 속도를 최대 3.0m/s로 제한
(4) 목표 조향각 절댓값 0.18rad 초과: 목표 속도를 최대 2.4m/s로 제한
(5) 목표 지점까지 거리 0.7m 미만: 목표 속도를 0m/s로 설정
(6) 주행 시간 35초 이상: 최종 정지 명령 적용

각 조건은 if문으로 독립적으로 검사한다. 따라서 하나의 조건을 만족하더라도 다음 조건을 계속 검사하며, 여러 제한이 동시에 적용될 수 있다.
특히 목표점까지의 거리가 0.7m 미만이면 GOAL_BRAKE 상태로 전환하고, 35초 이상이면 FINAL_BRAKE 상태로 전환한다.

```
if distance < 0.7:
    cruise = 0.0
    rule = "GOAL_BRAKE"

if t >= DRIVE_END:
    cruise = 0.0
    rule = "FINAL_BRAKE"
```

35초 이후에는 앞선 조건과 관계없이 cruise가 최종적으로 0이 되도록 설정한다.

⑨ 가속, 감속 변화율 제한

목표 속도가 결정되면 헌재 제어 속도인 smooth_speed와 비교하여 가속 또는 감속을 수행한다.

```
if cruise > smooth_speed:
    max_change = MAX_ACCEL * DT
else:
    max_change = MAX_DECEL * DT

smooth_speed = move_toward(
    smooth_speed,
    cruise,
    max_change
)
```

가속도 제한은 1.5m/s^2, 감속도 제한은 2.5m/s^2로 설정한다.

이 과정에서 목표 속도가 갑자기 변하더라도 제어 속도를 한 번에 변경하지 않고, 각 시간 간격에서 허용된 변화량만큼 조절한다.

⑩ 자동차 바퀴 제어

계산된 조향각과 제어 속도를 실제 자동차 바퀴에 적용한다,

```
wheel_target = smooth_speed / WHEEL_RADIUS
```

바쿠의 목표 각속도는 다음과 같이 계산한다.

$$w = \frac{v}{r}$$

여기서 v는 제어 속도, r은 바퀴 반지름 0.125m이다.

조향 바퀴에는 POSITION_CONTROL, 구동 바퀴에는 VELOCITY_CONTROL을 적용한다,

```
steering_joints = [4, 6]
drive_joints = [2, 3, 5, 7]
```

이렇게 계산된 조향각과 바퀴 회전 속도를 PyBullet 모터 제어 명령에 전달하여 자동차를 주행시킨다.

⑪ 중앙 영역 통과 및 정지 판정
자동차가 지정된 중앙 영역에 진입했는지 확인한다.

```
if (
    42 <= xy[0] <= 58
    and 42 <= xy[1] <= 58
    and passed_at is None
    and t <= DRIVE_END
):
    passed_at = t
```

35초 이내에 중앙 영역에 처음 진입하면 해당 시간을 passed_at에 기록한다.
이후 자동차의 속도와 바퀴 접지 상태를 이용하여 정지 여부를 판정한다.
정지 판정 조건은 다음과 같다,
- 주행 시간 31초 이상
- 실제 차량 속도 0.2m/s 이하
- 구동 바퀴 중 최소 1개가 지형에 접촉
- 위 조건이 0.5초 이상 연속 유지

조건을 충족하면 정지 시작 시간을 stopped_at에 기록한다. 조건이 중간에 해제되면 연속 정지 시간 측정을 초기화한다.

⑫ 주행 데이터 기록 및 화면 갱신
주행 과정에서 측정한 데이터를 일정한 간격으로 기록한다.

기능	실행 간격	내용
CSV 데이터 수집	12 step (0.1s)	위치, 속도, 가속도, 자세, 경사각 등
콘솔 출력	120 step (1s)	시간, 위치, 속도, 목표 거리, 경사각, 주행 상태
카메라 갱신	3 step (0.025s)	자동차 위치와 진행 방향 추적
상태 텍스트	24 step (0.2s)	시간, 속도, 목표 거리, 제어 상태 표시

CSV 데이터는 반복문 안에서 rows 배열에 저장하고, 시뮬레이션 반복이 종료된 후 실제 CSV 파일로 기록한다.

이후 다음 코드를 통해 물리 계산을 수행한다.

```
p.stepSimulation()
```

물리 계산을 완료하면 다음 step으로 이동하여 위치 측정부터 제어 과정을 반복한다.

⑬ CSV 저장 및 최종 결과 평가
40초까지 반복문을 실행한 후 수집된 주행 데이터를 다음 파일에 저장한다.
hw2_bigcar_smooth_log.csv
이후 최종 위치와 속도를 측정하고 다음 네 가지 조건을 평가한다.

```
passed_at is not None
stopped_at is not None
final_distance <= 1.0
final_speed <= 0.2
```

각 조건은 중앙 영역 통과 여부, 정지 확인 여부, 목표 위치 1m 이내 도달 여부, 최종 속도 0.2m/s 이하 여부를 의미한다.
마지막으로 결과를 콘솔에 출력하고, p.disconnect()를 실행하여 PuBullet 연결을 종료한다.

3. 실제 시뮬레이션 결과와 순서도 비교
제공한 실행 로그를 기준으로 순서도의 주요 단계가 실제로 수행되었는지 확인한 결과는 다음과 같다.

순서도 단계	실제 실행 결과
곡면 지형 및 자동차 생성	정상 실행
자동차 착지 안정화	READY 출력
목표 방향에 따른 주행	목표점 방향으로 이동
경사도 속도 제한	SLOPE_LIMIT 기록
중앙 영역 통과 판정	29.37초
최종 감속 및 정지	35.03초 정지 확인
목표 위치 오차 계산	0.28m
최종 속도 판정	0.00m/s
CSV 파일 저장	정상 저장
최종 조건 평가	4개 조건 모두 True

단, 실행 로그에서 기록된 SLOPE_LIMIT는 해당 순간 경사도 제한 조건이 적용되었다는 의미이며, 실제 속도가 정확히 3.0m/s로 유지되었다는 뜻은 아니다.

4. 시뮬레이션 순서도 및 코드 설명
본 시뮬레이션은 PyBullet 물리 엔진을 기반으로 곡면 지형에서 자동차가 목표 지점까지 자동으로 주행하고 정지하도록 구현하였다.
먼저 PyBullet GUI와 물리 환경을 초기화하고, 삼각함수 기반의 높이 함수를 이용하여 곡면 지형을 생성하였다. 이후 목표 영역을 설정하고 자동차 모델을 생성한 뒤, 360회의 물리 계산을 수행하여 차량의 초기 착지 상태를 안정화하였다.
주행이 시작되면 자동차의 현재 위치, 속도 및 회전 자세를 측정하고, 목표 지점까지의 거리와 방향을 계산한다. 계산된 방향 오차를 이용하여 목표 조향각을 결정하며, 조향각을 변화율을 제한하여 급격한 방향 전환을 방지하도록 하였다.
다음으로 지형의 기울기를 이용하여 경사각을 계산하고, 목표 쩜가지의 거리와 남은 주행 시간을 바탕으로 목표 속도를 결정한다. 출발 후 4초 동안은 목표 속도를 점진적으로 증가시키고, 32초 이후에는 남은 시간과 거리를 고료하여 속도를 제한한다. 또한 경사각이 6도를 초과하면 모교 속도를 최대 3.0m/s로 제한하고, 목표 조향각의 절댓값이 0.18rad을 초과하면 최

대 2.4m/s로 제한한다.
목표 지점까지의 거리가 0.7m 미만이거나 주행 시간이 35초 이상이 되면 목표 속도를 0으로 설정한다. 이때 가속도와 감속도의 변화율을 제한하여 제어 속도가 급격하게 변하지 않도록 하였으며, 계산된 조향각과 바퀴 화전 속도를 자동차의 모터에 적용하였다.
주행 중에는 중앙 영역 통과 여부를 확인하고, 차량 속도가 0.2m/s 이하이면서 바퀴가 지형에 접축한 상태가 0.5초 이상 유지되는지를 확인하여 정지 여부를 판정하였다.
이러한 과정을 1/120초 간격으로 반복하면서 차량의 위치, 속도, 가속도, 경사각 및 주행 상태 등을 기록하였다. 전체 40초의 시뮬레이션이 종료되면 수집된 데이터를 CSV 파일로 저장하고, 중앙 영역 통과 여부, 정지 여부, 목표 위치 오차 및 최종 속도를 평가하였다.
실제 실행 결과 자동차는 29.37초에 중앙 영역에 최초 진입하였고, 35.03초에 정지가 확인되었다. 최종 목표 위치 오차는 0.28m, 최종 속도 0.00m/s로 나타나 설정된 네 가지 최종 판정 조건을 모두 충족하였다.

사용 AI: Chat gpt plus, Google gemini, Deepseek4.1

교수님께 드리는 말씀

이번 과제를 수행하면서 제공된 OBJ 파일을 활용하여 자동차가 곡면 지형을 주행하는 시뮬레이션을 구현하고자 하였습니다. 처음에는 Blender 프로그램을 이용하여 OBJ 파일의 지형을 확인하였고, 이후 AI의 도움을 받아 Python 코드를 작성하였습니다. 그러나 코드를 실행하는 과정에서 지형 파일을 정상적으로 불러오지 못하는 문제와 여러 오류가 발생하였고, 반복적으로 수정하고 실행해 보았지만 끝내 해결하지 못했습니다.

AI를 활용하여 코드를 작성하더라도 작성된 코드의 구조와 동작 원리를 어느 정도 이해하고 있어야 오류를 해결할 수 있다는 것을 이번 과제를 통해 느꼈습니다. 하지만 저는 아직 Python의 기초 문법조차 충분히 이해하지 못하는 수준이었기 때문에 발생한 오류의 원인을 정확하게 파악하고 해결하는 데 어려움이 있었습니다.

결과적으로 교수님께서 제시하신 OBJ 파일을 활용한 시뮬레이션이라는 과제 조건을 충족하지 못했습니다. 그렇지만 과제를 아예 제출하지 않는 것보다는 제가 할 수 있는 범위에서라도 결과물을 만들어 제출하는 것이 맞다고 생각하였습니다. 따라서 제공된 OBJ 파일 대신 직접 곡면 지형을 생성하고, 자동차가 목표 지점까지 약 40초 동안 주행하는 시뮬레이션을 구현하였습니다.
비록 이번 과제에서 요구된 조건을 모두 충족하지 못했지만, 시뮬레이션을 구현하는 과정에서 PyBullet의 활용 방법과 자동차의 주행 제어에 대해 조금이나마 배울 수 있었습니다. 또한 AI가 작성한 코드를 단순히 실행하는 것과 그 코드를 직접 이해하고 수정하는 것은 전혀 다르다는 사실도 깨달았습니다.

다음 과제에서는 Python의 기초 문법과 코드 구조를 더욱 공부하여, AI의 도움을 받더라도 작성된 코드를 스스로 이해하고 오류를 해결할 수 있도록 노력하겠습니다. 이번 과제를 요구 조건에 맞게 완성하지 못하여 죄송합니다.
부족한 결과물이지만 끝까지 읽어주셔서 감사합니다.

[AI와 대화내용]
나: 압축을 풀어 OBJ 파일 2개를 읽으라고 명령
AI: OBJ 파일의 텍스트 파일을 읽어서 알려줌

나: 보고서 조건을 알려주고 코드를 만들어주라고 요청함
AI: 코드를 만들어줌

나: OBJ 파일을 사용한 코드에서 error가 떠서 그것을 캡쳐해서 질문함
AI: 수정 방법과 코드를 다시 만들어줌

나: 수정해준 것을 실행하였지만 계속된 오류로 시뮬레이션을 실패함

나: 과제의 조건을 충족하지 못하고 내가 울록볼록한 곡면경사 경로를 만들어달라고 요청함
AI: 만들어주어서 실행하였더니 맞게 실행됨

나: 만들어준 코드를 바탕으로 순서도, 코드 설명을 요청함
AI: 코드와 순서도의 일치를 설명해주고, 수식을 설명해줌

[붙임-로그 기록]

시간(s)	X좌표(m)	Y좌표(m)	속도(m/s)	목표까지 거리(m)	경사각	주행 상태
0	1.93	1.88	1.38	65.19	17.2	SLOPE_LIMIT
1	2.27	2.23	1.12	64.70	16.5	SLOPE_LIMIT
2	3.44	3.43	2.21	63.02	14.4	SLOPE_LIMIT
3	5.00	4.98	2.23	60.82	15.0	SLOPE_LIMIT
4	6.54	6.51	2.20	58.65	18.1	SLOPE_LIMIT
5	8.05	8.02	2.18	56.52	19.5	SLOPE_LIMIT
6	9.54	9.54	2.17	54.39	17.9	SLOPE_LIMIT
7	11.04	11.07	2.16	52.25	14.9	SLOPE_LIMIT
8	12.55	12.60	2.13	50.10	14.4	SLOPE_LIMIT
9	14.04	14.10	2.13	47.98	17.2	SLOPE_LIMIT
10	15.50	15.58	2.09	45.91	19.3	SLOPE_LIMIT
11	16.94	17.03	2.12	43.86	18.8	SLOPE_LIMIT
12	18.39	18.49	2.10	41.80	16.1	SLOPE_LIMIT
13	19.87	19.96	2.10	39.72	14.0	SLOPE_LIMIT
14	21.35	21.43	2.07	37.64	15.7	SLOPE_LIMIT
15	22.79	22.87	2.06	35.60	18.6	SLOPE_LIMIT
16	24.20	24.29	2.05	33.59	19.4	SLOPE_LIMIT
17	25.61	25.71	2.05	31.60	17.7	SLOPE_LIMIT
18	27.02	27.13	2.03	29.59	14.9	SLOPE_LIMIT
19	28.43	28.55	1.99	27.59	14.3	SLOPE_LIMIT
20	29.81	29.95	1.96	25.62	16.8	SLOPE_LIMIT

21	31.16	31.31	1.96	23.71	19.1	SLOPE_LIMIT
22	32.49	32.65	1.94	21.82	19.2	SLOPE_LIMIT
23	33.83	33.98	1.91	19.94	17.2	SLOPE_LIMIT
24	35.17	35.31	1.91	18.05	14.6	SLOPE_LIMIT
25	36.50	36.64	1.87	16.16	14.4	SLOPE_LIMIT
26	37.82	37.93	1.86	14.32	16.8	SLOPE_LIMIT
27	39.09	39.19	1.82	10.78	19.0	SLOPE_LIMIT
28	40.34	40.42	1.79	9.05	19.4	SLOPE_LIMIT
29	41.56	41.64	1.75	7.35		SLOPE_LIMIT
30	42.76	42.84	1.70	5.68	17.8	SLOPE_LIMIT
31	43.95	44.02	1.65	4.25	15.3	SLOPE_LIMIT
32	44.96	45.03	1.42	2.84	14.0	SLOPE_LIMIT
33	45.95	46.03	1.41	1.44	15.0	SLOPE_LIMIT
34	46.96	47.01	1.43	0.30	17.0	SLOPE_LIMIT
35	47.78	47.80	0.28	0.28	18.8	FINAL_BRAKE
36	47.79	47.81	0.00	0.28	19.4	FINAL_BRAKE
37	47.79	47.81	0.00	0.28	19.4	FINAL_BRAKE
38	47.79	47.81	0.00	0.28	19.4	FINAL_BRAKE
39	47.79	47.81	0.00	0.28	19.4	FINAL_BRAKE
40	47.79	47.81	0.00	0.28	19.4	FINAL_BRAKE

[붙임-코드]

```
import csv
import math
import time
from pathlib import Path

import numpy as np
import pybullet as p
import pybullet_data


# ============================================================
# 1. 기본 설정
# ============================================================

DT = 1.0 / 120.0

DRIVE_END = 35.0
DISPLAY_END = 40.0

SIZE = 100.0
N = 201

A = 1.8
WAVELENGTH = 32.0

START = np.array([3.0, 3.0], dtype=float)
GOAL = np.array([48.0, 48.0], dtype=float)

CAR_SCALE = 2.5

# 부드러운 속도 및 조향 제어
MAX_ACCEL = 1.5
MAX_DECEL = 2.5
MAX_STEER_RATE = 0.65

MAX_SPEED = 4.5
MAX_STEER = 0.35

WHEEL_RADIUS = 0.125

# 카메라 갱신: 120 / 3 = 40Hz
CAMERA_INTERVAL = 3
CAMERA_SMOOTH = 0.16
CAMERA_YAW_SMOOTH = 0.12

# CSV 기록: 120 / 12 = 10Hz
LOG_INTERVAL = 12
```

```
# 콘솔 출력: 1초마다
PRINT_INTERVAL = 120


# ============================================================
# 2. 곡면 지형 함수
# ============================================================

def height(x, y):
    k = 2.0 * math.pi / WAVELENGTH
    return A * math.sin(k * x) * math.cos(k * y)


def gradient(x, y):
    k = 2.0 * math.pi / WAVELENGTH

    dzdx = A * k * math.cos(k * x) * math.cos(k * y)
    dzdy = -A * k * math.sin(k * x) * math.sin(k * y)

    return dzdx, dzdy


def wrap(angle):
    return (angle + math.pi) % (2.0 * math.pi) - math.pi


def move_toward(current, target, max_delta):
    return current + float(
        np.clip(target - current, -max_delta, max_delta)
    )


# ============================================================
# 3. 시뮬레이션 시작
# ============================================================

def main():

    cid = p.connect(p.GUI)

    if cid < 0:
        raise RuntimeError("PyBullet GUI 연결 실패")

    p.setAdditionalSearchPath(pybullet_data.getDataPath())

    p.setGravity(0, 0, -9.81)
    p.setTimeStep(DT)

    p.setPhysicsEngineParameter(
        numSolverIterations=60,
        fixedTimeStep=DT
```

```
)

p.configureDebugVisualizer(
    p.COV_ENABLE_SHADOWS, 0
)

p.configureDebugVisualizer(
    p.COV_ENABLE_GUI, 0
)

# ====================================================
# 4. 곡면 Heightfield 생성
# ====================================================

xs = np.linspace(0, SIZE, N)
ys = np.linspace(0, SIZE, N)

zdata = [
    height(float(x), float(y))
    for y in ys
    for x in xs
]

terrain_shape = p.createCollisionShape(
    shapeType=p.GEOM_HEIGHTFIELD,
    meshScale=[
        SIZE / (N - 1),
        SIZE / (N - 1),
        1.0
    ],
    heightfieldData=zdata,
    numHeightfieldRows=N,
    numHeightfieldColumns=N
)

terrain = p.createMultiBody(
    baseMass=0,
    baseCollisionShapeIndex=terrain_shape
)

p.resetBasePositionAndOrientation(
    terrain,
    [SIZE / 2, SIZE / 2, 0],
    [0, 0, 0, 1]
)

p.changeDynamics(
    terrain,
    -1,
    lateralFriction=1.3,
    restitution=0.0
```

```
)

p.changeVisualShape(
    terrain,
    -1,
    rgbaColor=[0.40, 0.64, 0.42, 1]
)

# ====================================================
# 5. 중앙 영역 및 목표점 표시
# ====================================================

corners = [
    [42, 42],
    [58, 42],
    [58, 58],
    [42, 58]
]

for i in range(4):

    a = corners[i]
    b = corners[(i + 1) % 4]

    p.addUserDebugLine(
        [a[0], a[1], height(*a) + 0.3],
        [b[0], b[1], height(*b) + 0.3],
        [1, 0.55, 0],
        3
    )

p.addUserDebugText(
    "GOAL",
    [GOAL[0], GOAL[1], height(*GOAL) + 1],
    [1, 0, 0],
    1.5
)

p.addUserDebugLine(
    [START[0], START[1], height(*START) + 0.3],
    [GOAL[0], GOAL[1], height(*GOAL) + 0.3],
    [0, 0.7, 1],
    2
)

# ====================================================
# 6. 큰 자동차 생성
# ====================================================

yaw0 = math.atan2(
    GOAL[1] - START[1],
```

```
    GOAL[0] - START[0]
)

quat = p.getQuaternionFromEuler(
    [0, 0, yaw0]
)

car = p.loadURDF(
    "racecar/racecar.urdf",
    [
        START[0],
        START[1],
        height(*START) + 1.3
    ],
    quat,
    globalScaling=CAR_SCALE
)

steering_joints = [4, 6]
drive_joints = [2, 3, 5, 7]

print("\n=== CAR JOINTS ===")

for i in range(p.getNumJoints(car)):
    print(
        i,
        p.getJointInfo(car, i)[1].decode()
    )

# 자동차 바퀴 접지력
for j in drive_joints:

    p.changeDynamics(
        car,
        j,
        lateralFriction=1.4,
        restitution=0.0
    )

    p.setJointMotorControl2(
        car,
        j,
        p.VELOCITY_CONTROL,
        targetVelocity=0,
        force=0
    )

# ======================================================
# 7. 자동차 착지 안정화
# ======================================================
```

```
print("\n자동차 착지 안정화 중...")

for _ in range(360):
    p.stepSimulation()

pos, orn = p.getBasePositionAndOrientation(car)

print(
    "READY:",
    tuple(round(v, 3) for v in pos)
)

# ======================================================
# 8. 주행 변수 초기화
# ======================================================

smooth_steer = 0.0
smooth_speed = 0.0

camera_target = np.array(pos, dtype=float)
camera_yaw = math.degrees(yaw0) - 90.0

passed_at = None
stopped_at = None
stop_since = None

max_roll = 0.0
total_distance = 0.0

last_speed = 0.0

previous_xy = np.array(pos[:2], dtype=float)

rows = []

debug_id = -1

path = Path("hw2_bigcar_smooth_log.csv")

print("\n=== SIMULATION START ===")

# ======================================================
# 9. 주행 반복
# ======================================================

for step in range(int(DISPLAY_END / DT) + 1):

    loop_start = time.perf_counter()

    t = step * DT
```

```
        pos, orn = p.getBasePositionAndOrientation(car)
        vel, _ = p.getBaseVelocity(car)

        roll, pitch, yaw = p.getEulerFromQuaternion(orn)

        xy = np.array(pos[:2], dtype=float)

        speed = float(np.linalg.norm(vel))

        total_distance += float(
            np.linalg.norm(xy - previous_xy)
        )

        previous_xy = xy.copy()

        max_roll = max(
            max_roll,
            abs(math.degrees(roll))
        )

        distance = float(
            np.linalg.norm(GOAL - xy)
        )

        # ================================================
        # 10. 목표 방향 계산
        # ================================================

        heading = math.atan2(
            GOAL[1] - xy[1],
            GOAL[0] - xy[0]
        )

        err_heading = wrap(heading - yaw)

        target_steer = float(
            np.clip(
                1.6 * err_heading,
                -MAX_STEER,
                MAX_STEER
            )
        )

        # 조향각 변화량 제한
        smooth_steer = move_toward(
            smooth_steer,
            target_steer,
            MAX_STEER_RATE * DT
        )

        # ================================================
```

```
        # 11. 경사각 계산
        # ================================================

        dzdx, dzdy = gradient(
            xy[0],
            xy[1]
        )

        slope = math.degrees(
            math.atan(
                math.hypot(dzdx, dzdy)
            )
        )

        # ================================================
        # 12. 목표속도 계산
        # ================================================

        remaining_time = max(
            0.1,
            DRIVE_END - t
        )

        cruise = min(
            MAX_SPEED,
            distance / remaining_time * 1.15
        )

        # 출발 시 서서히 가속
        if t < 4.0:
            cruise = min(
                cruise,
                MAX_SPEED * t / 4.0
            )

        # 31초 이후 감속
        if t >= 31.0:
            cruise = min(
                cruise,
                distance / remaining_time
            )

        rule = "NORMAL"

        # 경사 제한
        if slope > 6.0:

            cruise = min(
                cruise,
                3.0
            )
```

```
        rule = "SLOPE_LIMIT"

    # 회전 제한
    if abs(target_steer) > 0.18:

        cruise = min(
            cruise,
            2.4
        )

        rule += "+TURN_LIMIT"

    # 목표점 근처 감속
    if distance < 0.7:

        cruise = 0.0
        rule = "GOAL_BRAKE"

    # 35초 이후 정지 명령
    if t >= DRIVE_END:

        cruise = 0.0
        rule = "FINAL_BRAKE"

    # ================================================
    # 13. 부드러운 가속 및 감속
    # ================================================

    if cruise > smooth_speed:
        max_change = MAX_ACCEL * DT
    else:
        max_change = MAX_DECEL * DT

    smooth_speed = move_toward(
        smooth_speed,
        cruise,
        max_change
    )

    smooth_speed = max(
        0.0,
        smooth_speed
    )

    # ================================================
    # 14. 자동차 바퀴 제어
    # ================================================

    wheel_target = (
        smooth_speed / WHEEL_RADIUS
```

```
        )

        for j in steering_joints:

            p.setJointMotorControl2(
                car,
                j,
                p.POSITION_CONTROL,
                targetPosition=float(smooth_steer),
                force=60
            )

        for j in drive_joints:

            p.setJointMotorControl2(
                car,
                j,
                p.VELOCITY_CONTROL,
                targetVelocity=float(wheel_target),
                force=35.0 if smooth_speed > 0.01 else 80.0
            )

        # ================================================
        # 15. 중앙 영역 통과 판정
        # ================================================

        if (
            42 <= xy[0] <= 58
            and 42 <= xy[1] <= 58
            and passed_at is None
            and t <= DRIVE_END
        ):

            passed_at = t

            print(
                f"CENTER FIRST PASS: {t:.2f}s"
            )

        # ================================================
        # 16. 바퀴 접지 및 정지 판정
        # ================================================

        contact_count = sum(
            bool(
                p.getContactPoints(
                    car,
                    terrain,
                    linkIndexA=j
                )
            )
```

```
            for j in drive_joints
        )

        if (
            t >= 31.0
            and speed <= 0.2
            and contact_count >= 1
        ):

            if stop_since is None:
                stop_since = t

            if (
                stopped_at is None
                and t - stop_since >= 0.5
            ):

                stopped_at = stop_since

                print(
                    f"STOP CONFIRMED: {stopped_at:.2f}s"
                )

        else:
            stop_since = None

        # ================================================
        # 17. CSV 데이터 기록
        # ================================================

        if step % LOG_INTERVAL == 0:

            accel = (
                (speed - last_speed)
                / (LOG_INTERVAL * DT)
                if step else 0.0
            )

            last_speed = speed

            rows.append([
                round(t, 3),
                round(pos[0], 3),
                round(pos[1], 3),
                round(pos[2], 3),
                round(speed, 3),
                round(accel, 3),
                round(math.degrees(roll), 2),
                round(math.degrees(pitch), 2),
                round(math.degrees(yaw), 2),
                round(smooth_steer, 3),
```

```
                round(slope, 2),
                round(smooth_speed, 3),
                round(distance, 3),
                contact_count,
                rule,
                int(passed_at is not None)
            ])

        # ================================================
        # 18. 콘솔 출력
        # ================================================

        if step % PRINT_INTERVAL == 0:

            print(
                f"t={t:5.1f}s "
                f"pos=({xy[0]:6.2f},{xy[1]:6.2f}) "
                f"speed={speed:.2f}m/s "
                f"dist={distance:.2f}m "
                f"slope={slope:.1f}deg "
                f"{rule}"
            )

        # ================================================
        # 19. 부드러운 카메라 추적
        # ================================================

        if step % CAMERA_INTERVAL == 0:

            camera_target += CAMERA_SMOOTH * (
                np.array(pos) - camera_target
            )

            desired_yaw = (
                math.degrees(yaw) - 90.0
            )

            yaw_difference = (
                (desired_yaw - camera_yaw + 180)
                % 360 - 180
            )

            camera_yaw += (
                CAMERA_YAW_SMOOTH * yaw_difference
            )

            p.resetDebugVisualizerCamera(
                cameraDistance=9,
                cameraYaw=camera_yaw,
                cameraPitch=-35,
                cameraTargetPosition=camera_target.tolist()
```

```
            )

        # ================================================
        # 20. 상태 텍스트
        # ================================================

        if step % 24 == 0:

            debug_id = p.addUserDebugText(
                (
                    f"Time: {t:.1f}s | "
                    f"Speed: {speed:.2f}m/s | "
                    f"Goal: {distance:.1f}m | "
                    f"{rule}"
                ),
                [
                    pos[0],
                    pos[1],
                    pos[2] + 1.0
                ],
                [1, 1, 0],
                1.2,
                lifeTime=0.3,
                replaceItemUniqueId=debug_id
            )

        # ================================================
        # 21. 물리 계산 및 시간 동기화
        # ================================================

        p.stepSimulation()

        elapsed = time.perf_counter() - loop_start

        sleep_time = DT - elapsed

        if sleep_time > 0:
            time.sleep(sleep_time)

# ======================================================
# 22. CSV 파일 저장
# ======================================================

with path.open(
    "w",
    newline="",
    encoding="utf-8-sig"
) as f:

    writer = csv.writer(f)
```

```
        writer.writerow([
            "time_s",
            "x_m",
            "y_m",
            "z_m",
            "speed_mps",
            "accel_mps2",
            "roll_deg",
            "pitch_deg",
            "yaw_deg",
            "steer_rad",
            "slope_deg",
            "target_speed_mps",
            "goal_distance_m",
            "wheel_contacts",
            "rule",
            "center_passed"
        ])

        writer.writerows(rows)

# ======================================================
# 23. 최종 결과
# ======================================================

final_pos, _ = p.getBasePositionAndOrientation(car)

final_vel, _ = p.getBaseVelocity(car)

final_distance = float(
    np.linalg.norm(
        GOAL - np.array(final_pos[:2])
    )
)

final_speed = float(
    np.linalg.norm(final_vel)
)

print("\n================================")
print("          SIMULATION SUMMARY")
print("================================")

print("Center first pass:", passed_at)
print("Confirmed stop start:", stopped_at)

print(
    "Final goal error:",
    round(final_distance, 2),
    "m"
)
```

```
print(
    "Final speed:",
    round(final_speed, 3),
    "m/s"
)

print(
    "Total distance:",
    round(total_distance, 2),
    "m"
)

print(
    "Max abs roll:",
    round(max_roll, 2),
    "deg"
)

print("CSV:", path.resolve())

print("\n=== CONDITION CHECK ===")

print(
    "Center passed:",
    passed_at is not None
)

print(
    "Stop confirmed:",
    stopped_at is not None
)

print(
    "Goal within 1m:",
    final_distance <= 1.0
)

print(
    "Final speed <= 0.2m/s:",
    final_speed <= 0.2
)

print(
    "\nNOTE: 목표 도달 및 정지 조건은 "
    "실제 실행 결과를 확인해야 합니다."
)

print("\n시뮬레이션 종료")

p.disconnect()
```

```
if __name__ == "__main__":
    main()
```